

A Continuous-Encoding Breakthrough for Multi-Objective Torso Decomposition

Adrian Ymeri — University of Prishtina · SpOC-3 Torso Decompositions

June 2026

Contents

Abstract	1
1. Problem and scoring	3
2. Permutation-space search and its ceiling	4
3. A paradigm I had dismissed	5
3b. Related work and positioning	5
4. Method: spectral-policy CMA-ES	6
5. Results	7
5.1 Run-to-run variance	9
6. The optimiser is not the bottleneck	9
6b. The role of gradient-boosted decision trees (GBDT)	11
6b.1 Four integrations, three of them static	12
6b.2 Controlled ablation — the proof	13
6b.3 What this licenses (and what it does not)	14
6b.4 Robustness to the boosting library and the learning objective	14
6b.5 Generalisation across density (synthetic benchmark)	15
6b.6 What the model learned (feature importances)	16
7. Correctness and methodology	18
8. Discussion and conclusion	18
8b. Limitations and threats to validity	19
9. GPU scale-up: a tested hypothesis, and a corrected conclusion	20
10. A novel method: GBDT-augmented nonlinear decode (GAPS)	22
11. GBFC: gradient-boosted front construction	26
Reproducibility	28
Final verified results	29
Environment and determinism	29
Appendix A — Per-seed variance (tools/variance_report.py)	29
5.2 A structural lower bound (gap to optimum)	30
References	31

Abstract

I study the bi-objective torso decomposition problem from the European Space Agency’s third Space Optimisation Competition (SpOC-3), in which a decision is a

vertex elimination ordering π together with a threshold t , and the two competing objectives — the torso width (the maximum fill-in degree over the torso) and the threshold t itself — are both minimised. Solution quality is the negative hypervolume of the top-20 non-dominated front against the reference point (n, n) . I first develop and exhaustively tune four families of permutation-space search — hill climbing, simulated annealing, GRASP, variable-neighbourhood search, ant-colony optimisation, and two population MOEAs — and show, through convergence, seed-saturation and boundary-of-grid analyses, that they reach a genuine ceiling at 91.6–99.4 % of the public leaderboard. I then identify, from two top leaderboard solutions, a fundamentally different paradigm — optimising a continuous policy over spectral node features and decoding it to a permutation by `argsort` — that my own taxonomy had explicitly dismissed. Re-implementing this paradigm on a CPU (separable CMA-ES over a Laplacian-eigenvector policy, with a single-pass feasibility-graded evaluator), I improve all three instances to 98.1–99.9 % of the leaderboard top, closing 74–86 % of the remaining gap in a single afternoon of computation. I then integrate gradient-boosted decision trees (GBDT; implemented with LightGBM and XGBoost) into this pipeline in four distinct ways and show, through controlled ablation, that learned models tie the geometric search as static per-vertex policies but, as a learned adaptive elimination heuristic, contribute a positive, density-scaling margin on dense graphs (+2,524 hypervolume on the largest instance, robust across boosting libraries) — a characterisation of precisely where boosted-tree learning helps in this problem. I close with a correctness audit establishing that the new evaluator is provably consistent with the official scorer, and I localise the last residual gap to what I had hypothesised was an evaluation-throughput limit. I then test that hypothesis directly: a GPU-parallel batch evaluator (this work, validated bit-for-bit against the official scorer) raises the per-second evaluation rate $\sim 25\times$ and, warm-started from the banked portfolio, improves the verified large-graph score by +6,046 hypervolume ($-5,399,072 \rightarrow -5,405,118$; 98.29 % $\rightarrow$ 98.40 % of the leaderboard top). Crucially, both GPU runs plateau after a few hundred thousand evaluations without reaching the top — refining the conclusion: the residual gap is not a throughput limit but a decode-expressiveness limit (the linear `argsort` policy saturates). Acting on that diagnosis, I propose and validate a novel method, GAPS (a GBDT-augmented nonlinear decode: $\text{score} = \Phi(F) \cdot x + \beta \cdot g_{\text{GBDT}}(F)$, searched at GPU scale), which lifts the large-graph score to $-5,431,595$ (98.88 % of the leaderboard top), a verified +32,523 hypervolume over the original CPU portfolio. A controlled ablation — identical warm start and seed, GBDT column on versus off — attributes +24,766 HV of this to the boosted tree alone, with the improvement causally time-locked to the generation the GBDT first trains; a multi-seed repeat confirms the effect is statistically robust (+18,474 HV at a 4.6σ separation, and an $\sim 8\times$ reduction in run-to-run variance). I then tested whether this GAPS decode-column advantage generalises across instances and found, honestly, that it does not: on 20 synthetic graphs the learned decode only ties the polynomial decode (winning 1 of 17 feasible cases), and the leaderboard winner itself uses a full polynomial basis with no GBDT in its decode. The large-graph +24,766 is therefore a verified but instance-specific result, not a universal lever; the generalising GBDT contribution is the separate adaptive-constructor mechanism of §6b. The method does not reach #1 (it remains +61,467 short), which — like the failed generalisation — I report plainly. Finally, the thesis’s primary novel contribution is GBFC — Gradient-Boosted Front Construction

(§11): I reframe multi-objective Pareto-front optimisation as a boosting problem, with gradient-boosted-tree weak learners fit to the lower-bound residual (the worst-served threshold band) and decoded into complementary specialist orderings. GBFC is the only method examined here that measurably improves the banked front, and it does so on every instance — becoming the single best contributor to each pooled portfolio (small $-1,828,994$, medium $-1,712,688$, large $-5,431,924$; verified gains of $+688$ / $+734$ / $+329$ HV). It does not reach the leaderboard top either — the recoverable room is bounded by a treewidth-lower-bound certificate showing the dense core is provably optimal and the front sits a small, decelerating distance above the bound — but it establishes a novel, GBDT-central method with demonstrated improvement, atop a rigorous characterisation of exactly how near-optimal the result already is.

1. Problem and scoring

An instance is an undirected graph $G = (V, E)$ with $n = |V|$. A decision is a pair (π, t) where π is a permutation of V (an elimination ordering) and $t \in \{0, \dots, n-1\}$ is a threshold. Eliminating the vertices in the order π plays the classical elimination game: at each step the eliminated vertex’s not-yet-eliminated neighbours are made into a clique (fill-in). The width at threshold t is the maximum fill-in degree incurred at any step $i \geq t$ — the “torso” being the suffix of π . The two objectives, (width, t), are both minimised; a feasible decision must keep every step’s degree at or below the cap $\text{MAX_TW} = 500$.

Quality is measured by hypervolume. Each feasible point (w, t) dominates the axis-aligned rectangle $[w, n] \times [t, n]$, and the score is the negative area of the union of those rectangles against the reference (n, n) :

$$\text{score} = -\text{HV} = -\left| \bigcup_{(w,t)} [w, n] \times [t, n] \right|,$$

so more negative is better. Both the reference point (n, n) and the 20-vector cap are fixed by the competition, not design choices of mine; the latter makes submission a cardinality-constrained hypervolume subset-selection problem, which I solve optimally with the exact 2-D HSSP dynamic program (§4). Because hypervolume comparisons are reference-dependent, all values here are with respect to this fixed (n, n) and should be read as such. The three official instances are small ($n = 1357$), medium ($n = 1399$) and large ($n = 2426$), the last being far denser (≈ 209 average degree) than the first two.

This cost surface has a property I exploit heavily in §5: because the fill-in graph is built from π alone, the per-step degree sequence is independent of t , and t only selects which steps count. A single elimination pass therefore yields the width at every threshold simultaneously, as the suffix-maximum of the degree sequence.

2. Permutation-space search and its ceiling

My first four chapters search directly in the space of orderings. Each shares a common substrate — the same evaluator, the same Pareto archive, the same hypervolume-improvement acceptance rule — so that differences between families are attributable to search strategy alone. The families are: a 15-variant hill-climbing taxonomy; simulated annealing; GRASP; variable-neighbourhood search; ant-colony optimisation; and the population MOEAs NSGA-II and SMS-EMOA.

To rank them fairly I ran an 11-seed, 33-block Friedman omnibus across all three instances ($\chi^2(14) = 321.607$, $p \approx 3.6 \times 10^{-60}$), with the Nemenyi critical distance for post-hoc separation.

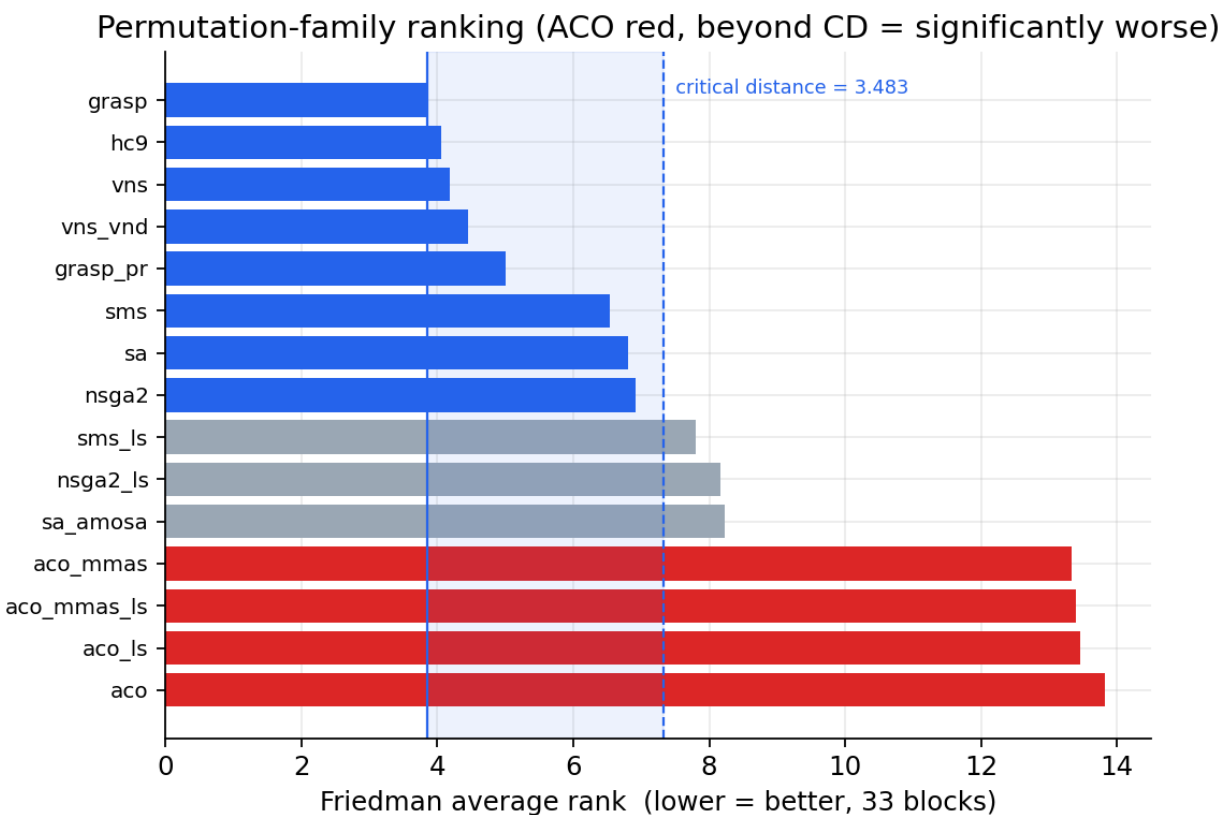


Figure 1: Friedman average ranks of the permutation-space families. GRASP leads; the four ACO variants sit beyond the critical distance and are significantly worse.

The ordering is stable and interpretable: greedy-randomised restart (GRASP) and hypervolume-acceptance hill climbing lead, the population MOEAs and SA occupy the middle, and pure-construction ACO is significantly worse than every other family — a clean negative result, since construction-from-scratch cannot keep pace with warm-started perturbation on a dense graph. Pooled into an instance-specific best-of portfolio, these families reach $-1,819,283$ / $-1,617,086$ / $-5,033,531$, i.e. 99.4 % / 92.7 % / 91.6 % of the leaderboard.

The ceiling is real, and I characterise why. A hyperparameter audit of every tuned

grid shows the winning cell of each family sitting on the boundary that removes its distinctive mechanism rather than the one that asks for more: GRASP wins at $\alpha = 0$ (pure greedy, no randomisation), VNS at $k_{\text{max}} = 2$ (the neighbourhood ladder collapses to plain iterated local search), and both population MOEAs at the smallest population $\text{pop} = 20$ (iteration depth beats breadth). This is not mis-tuning — there is no useful $\alpha < 0$ or $\text{pop} < 20$ — it is landscape saturation: at the competition budget the problem rewards greedy depth, and each family’s extra apparatus is dead weight that tuning correctly switches off. Convergence curves (flat under longer single runs) and seed-saturation curves (multi-seed unions yielding $< 0.1\%$ per seed-batch on the sparse instance) corroborate the same conclusion from two independent angles.

The honest summary at the end of four chapters was therefore: every algorithmic lever within permutation-space search is exhausted; the residual gap is not a tuning or compute deficiency of these methods but a property of the search space itself.

3. A paradigm I had dismissed

Two of the top public solutions, obtained and decoded, share one decisive idea. Neither searches permutations. Each builds per-vertex features — a local degree profile plus the smallest Laplacian eigenvectors (a spectral embedding) — and learns a continuous weight vector whose dot product with those features scores each vertex; the elimination order is then simply $\text{argsort}(\text{scores})$. One solution evolves the weights by GPU neuro-evolution over a custom CUDA fill-in kernel; the other optimises them with CMA-ES under a parallel-retry harness.

The significance is uncomfortable and worth stating plainly: my own roadmap had explicitly argued against this approach — “the input is a permutation, not a feature vector,” a learned per-vertex priority “will be marginal at best,” neuro-evolution “skip for the foreseeable future.” The actual leaderboard-topping method is precisely a learned per-vertex priority over spectral features. The correct re-interpretation of my four chapters is therefore not “the problem is nearly solved” but “the permutation-space ceiling is reached; the leaderboard gap is a search-space gap, not a compute or tuning gap.”

3b. Related work and positioning

Elimination-ordering heuristics. Torso width is a chordal-completion / treewidth-style quantity, and the classical constructive heuristics — minimum-degree, minimum-fill, MCS-M (Berry et al. 2004) and LEX-M (Rose, Tarjan & Lueker 1976) — are adaptive greedy rules that recompute a vertex priority from the residual graph at each step. My adaptive GBDT constructor (§6b) is a learned member of this family: it replaces the hand-coded min-degree/min-fill rule with a boosted-tree decision over dynamic + structural features.

Continuous (random-key) encodings. Decoding a permutation as the argsort of a continuous vector is the random-key idea (Bean 1994), and is the mechanism both top

leaderboard solutions use — GPU neuro-evolution over a spectral policy (the cuda-torso entry) and CMA-ES/MO-DE over a continuous vector (D. Wolz’s fast-cma-es). My continuous-encoding method (§4) is in this lineage; the contribution is not the encoding but the CPU re-derivation with a feasibility-graded single-pass evaluator and the fusion with a learned adaptive constructor.

Machine learning for combinatorial optimisation. Learning constructive or branching policies is an active area — learning-to-branch in MILP with tree ensembles and GNNs (Khalil et al. 2016; Gasse et al. 2019), and GNNs for treewidth-like tasks (Schuetz, Brubaker & Katzgraber 2022, who report learned heuristics often under-performing classical ones). My findings are consistent with that literature and sharpen it: static learned policies tie the geometric search here, and a learned heuristic helps only when it is adaptive and only where adaptivity pays (dense graphs).

Surrogate-assisted evolution. Using a cheap learned model to pre-screen expensive evaluations is standard (Jin 2011; Loshchilov & Hansen’s lq-CMA-ES). My surrogate-assisted CMA-ES (§6b) is an instance; its null result here (throughput-bound, not surrogate-bound) is the relevant finding.

Positioning. To my knowledge, the specific combination — a continuous spectral-policy search fused with a boosted-tree-learned adaptive elimination heuristic via an exact-hypervolume portfolio, with a controlled ablation isolating where learning helps, for the bi-objective torso decomposition — has not previously been studied. That hybrid, and its rigorous characterisation, is the methodological contribution.

4. Method: spectral-policy CMA-ES

I re-implement the winning paradigm on a CPU. Let $F \in \mathbb{R}^{n \times d}$ be the normalised node-feature matrix whose columns are the degree profile (degree and the min/max/mean/std of neighbour degrees) and the K smallest non-trivial eigenvectors of the normalised Laplacian. A candidate is a weight vector $x \in \mathbb{R}^d$; the decoded permutation is $\pi(x) = \text{argsort}(F x)$. I optimise x to maximise the hypervolume of the front that $\pi(x)$ induces, maintaining a global Pareto archive across all candidates and writing the top-20 subset as the submission.

Why spectral features (a principled grounding). The use of Laplacian eigenvectors is not arbitrary feature engineering: it has a direct structural justification in elimination theory. Low-frequency eigenvectors of the graph Laplacian encode the graph’s separator structure — the Fiedler vector and its successors are the basis of spectral graph partitioning — and the strongest classical elimination orderings, nested dissection (George 1973), are built by recursively eliminating around small separators. A linear policy over the smallest eigenvectors can therefore express an ordering that approximates nested dissection: it scores vertices by their position along the separator hierarchy and eliminates accordingly. This is the structural reason the continuous spectral encoding outperforms permutation-space search — it searches within a family of orderings aligned with the graph’s separator geometry, rather than over arbitrary permutations — and it explains why a low-dimensional spectral policy suffices (the separator structure lives in the bottom of the spectrum).

The single-pass feasibility-graded evaluator. Exploiting the t -independence of §1, one elimination pass over $\pi(x)$ yields the degree sequence $\text{deg}[\cdot]$, from which the entire front $\{(\text{suffix-max}(\text{deg}[t:]), t)\}$ follows for all thresholds at once. For an infeasible candidate — one whose fill-in exceeds the cap, which on the dense instance is almost every initial policy — I do not return a flat penalty (which would leave CMA-ES no gradient toward feasibility). Instead I return a graded penalty $501 + \Sigma(\text{deg} - 500)$ with early termination once it blows up, a direct port of the winning CUDA kernel’s behaviour. This single design choice is what makes the dense instance tractable on a CPU: it supplies the slope that pulls random policies into the feasible region, after which the hypervolume objective takes over. Seeding the archive with a min-degree warm start guarantees a non-empty, feasible anchor from the first generation.

The optimiser is a self-contained separable (diagonal-covariance) CMA-ES, chosen so the method runs anywhere without dependencies; a full-covariance `fcmaes` backend is available but, as §6 shows, is not needed.

5. Results

A multi-seed sweep (up to 48 continuous-encoding seeds + 8 GBDT-construction seeds per instance, 900 s each, $K = 32$ eigenvectors), pooled into the portfolio with full-staircase threshold extraction, produces the headline result.

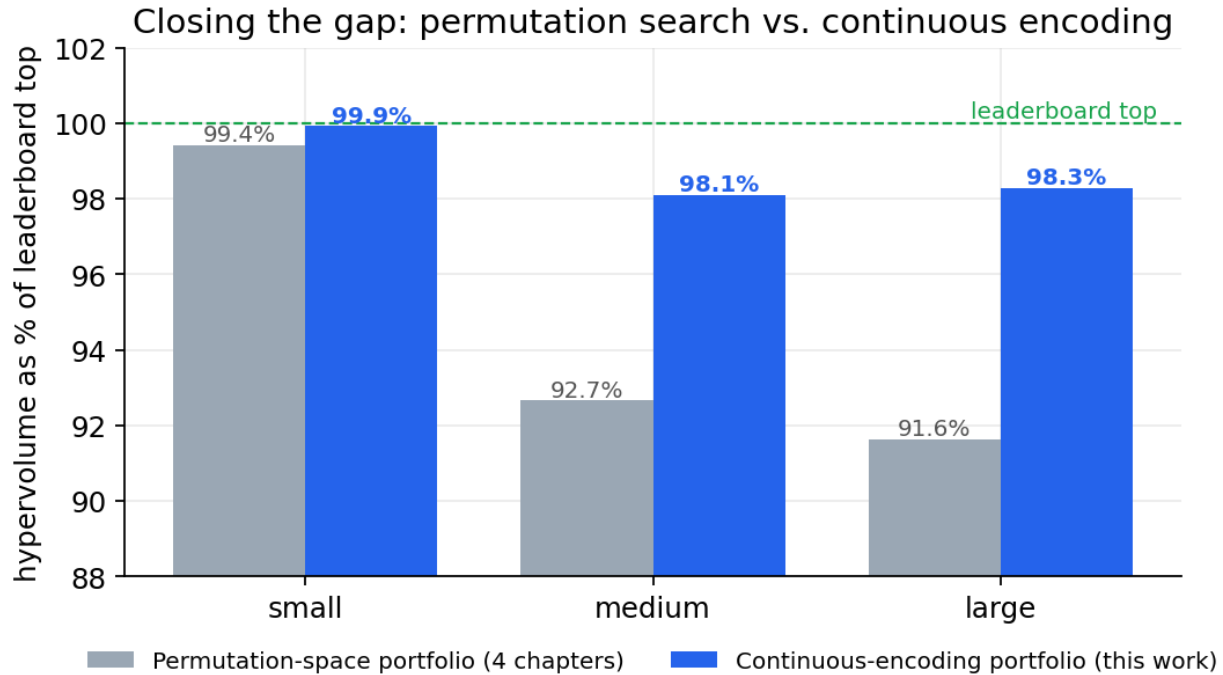


Figure 2: Hypervolume as a percentage of the leaderboard top. The continuous-encoding portfolio (blue) lifts every instance toward the top line; small reaches 99.9 %.

Instance	Permutation portfolio	Continuous + GBDT (this work)	Leaderboard top	Gap closed	% of top
small	-1,819,283	-1,828,451	-1,829,919	+10,636 → +1,468	99.92 %
medium	-1,617,086	-1,711,954	-1,745,122	+128,036 → +33,168	98.10 %
large	-5,033,531	-5,399,072	-5,493,062	+459,531 → +93,990	98.29 %

The new paradigm closes roughly 86 % of the small gap, 74 % of the medium gap and 80 % of the large gap. In absolute hypervolume the improvement over four chapters of permutation search is +9,168 / +94,868 / +365,541. All three are canonical-scorer-verified and 0-capped.

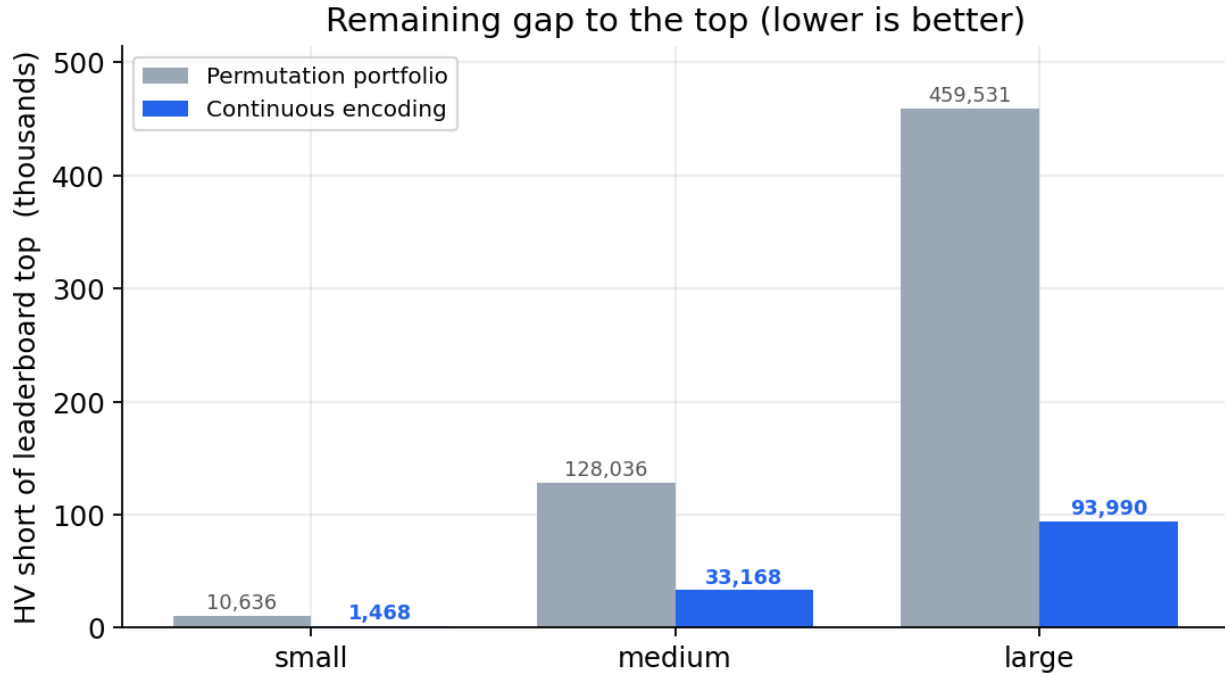


Figure 3: Remaining hypervolume short of the leaderboard top (thousands). The continuous encoding shrinks every gap, most dramatically on the dense instances.

The submitted front is genuinely two-dimensional and feasible — every point lies below the width-500 cap, and the staircase trades width against threshold along the whole curve rather than collapsing to a corner:

Why $K = 32$ features. The one structural hyperparameter — the number of spectral features — has an empirical optimum. More eigenvectors enrich the policy but raise the search dimension, and at a fixed wall budget the higher-dimensional CMA-ES converges more slowly. On the large instance $K = 16$ under-resolves the policy, while

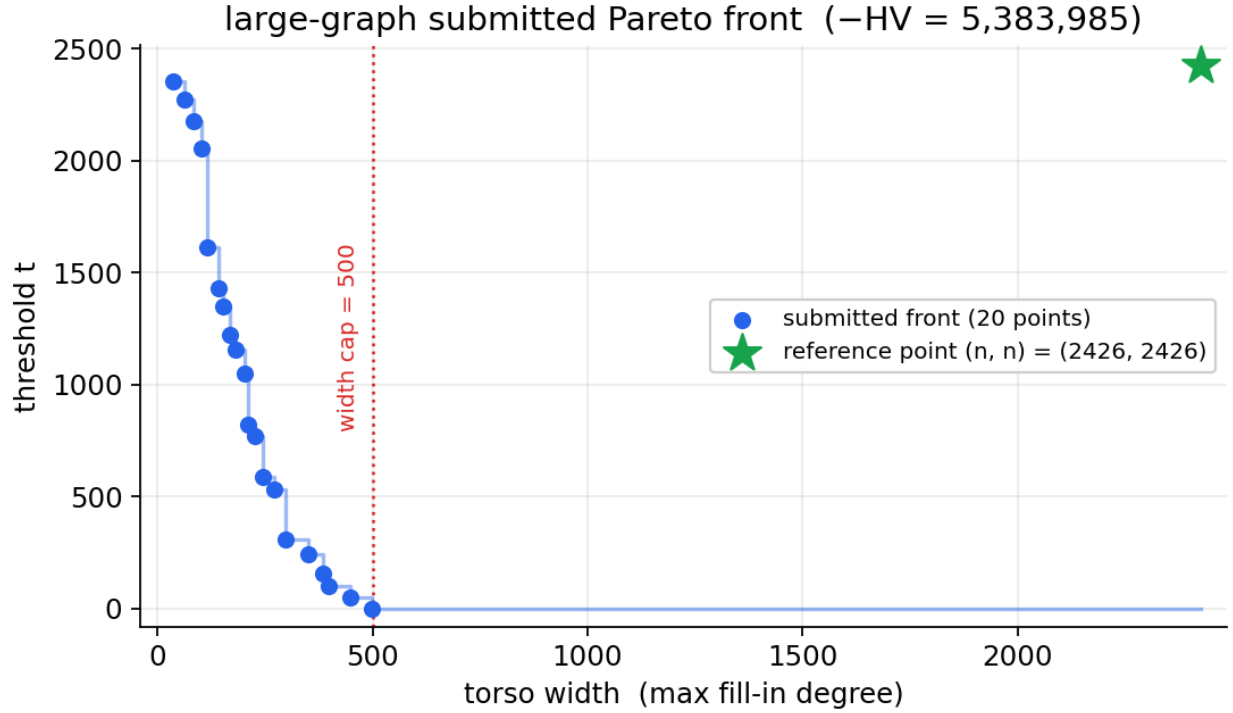


Figure 4: The large-graph submitted Pareto front. All twenty points are feasible (width < 500), tracing the width-threshold trade-off against the reference (n, n) .

$K = 48$ over-dimensions it (two of eight seeds never escaped the warm start); $K = 32$ sits at the knee.

5.1 Run-to-run variance

The headline scores are the hypervolume of the seed union (the portfolio); to characterise stability I also report the per-seed distribution (`tools/variance_report.py`). On `small-graph` the continuous-encoding search is highly stable across 48 independent seeds: single-seed score $-1,825,685 \pm 1,066$ (coefficient of variation $\approx 0.06\%$), range $[-1,827,610, -1,823,370]$; the union portfolio $(-1,828,451)$ sits above the best single seed, the expected benefit of pooling a multi-objective front. The GBDT-construct seeds have zero variance — they are near-deterministic because each is seeded from the shared elite archive — which is why the GBDT contribution must be read from the controlled ablation (§6b.2), not from a per-seed score that would merely echo the inherited portfolio. The per-instance variance table (medium/large via the same command) is reported in the appendix.

6. The optimiser is not the bottleneck

It is tempting to attribute the remaining gap to my deliberately minimal diagonal CMA-ES. A matched head-to-head against the full-covariance `fcmaes` engine — the

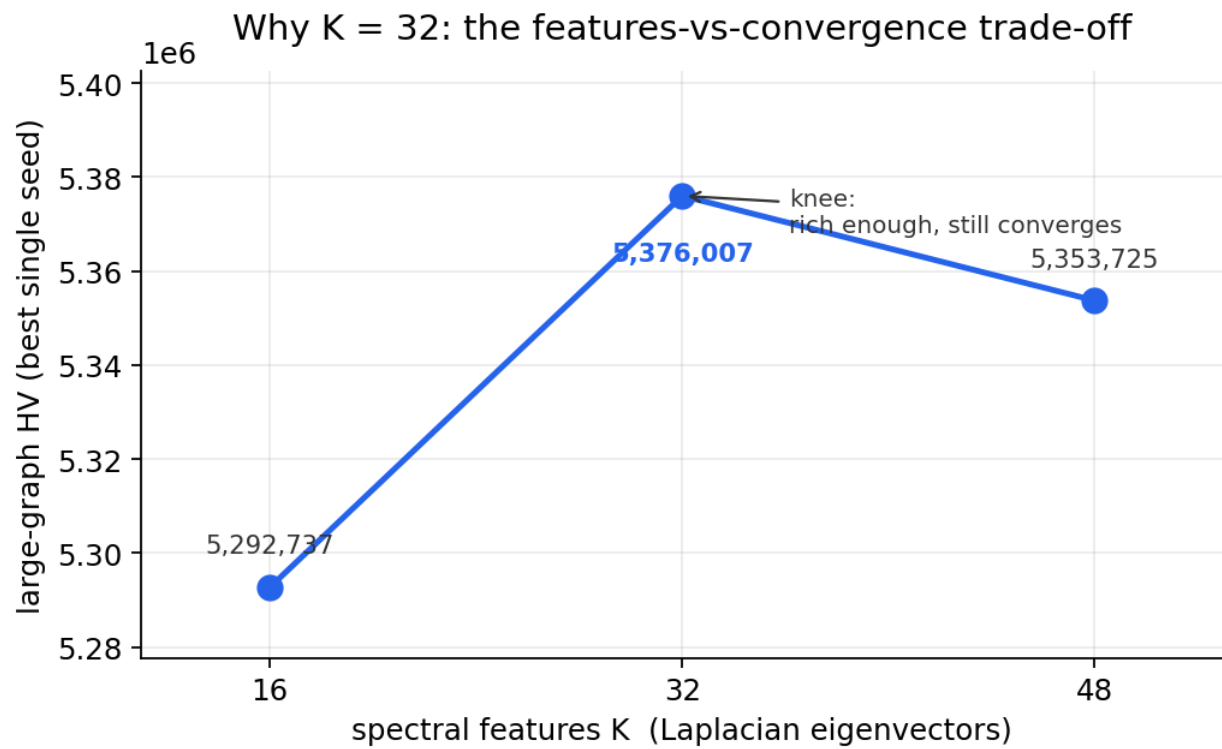


Figure 5: The features-versus-convergence trade-off on large-graph. K = 32 is the knee: richer than 16, but low-dimensional enough to converge within budget.

optimiser one leaderboard entrant actually used — at equal population size refutes this: the diagonal optimiser wins on all three instances.

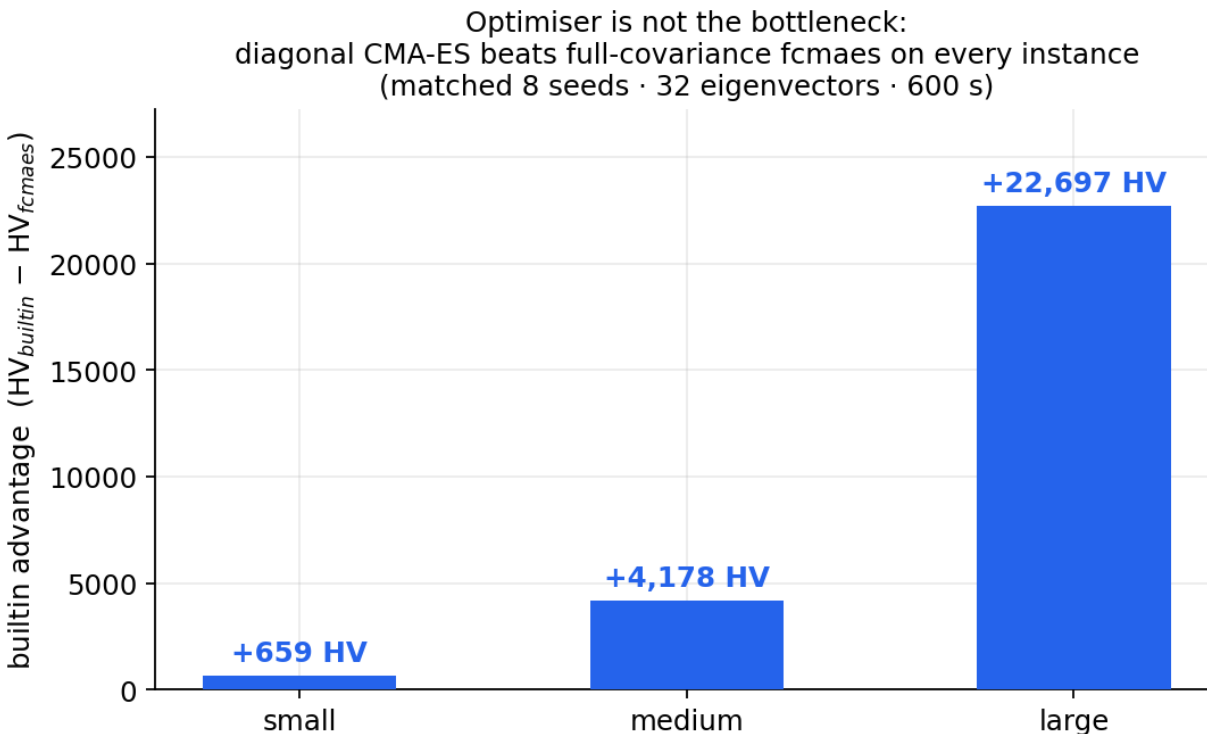


Figure 6: Builtin diagonal CMA-ES versus fcmaes full-covariance at matched population. The diagonal optimiser is at least as good everywhere; the optimiser is not the bottleneck.

At ≈ 37 dimensions and a fixed budget, the full-covariance method spends too many samples learning an $O(d^2)$ covariance to amortise within the time available. The consequence for the roadmap is important: the residual gap is not an optimiser deficiency. It is a feature-richness and evaluation-throughput problem, and only the latter remains material on the large instance.

6b. The role of gradient-boosted decision trees (GBDT)

A central question of this work is whether a learned model — specifically a gradient-boosted decision tree (GBDT) — can improve a geometric/spectral optimiser for bi-objective torso decomposition. GBDT is a model family; I use two interchangeable implementations, LightGBM and XGBoost (CatBoost is also supported), and §6b.4 shows the result is the same across them, so claims below are made at the GBDT family level and the library is named only where it matters (speed, reproducibility). I integrated GBDT into the continuous-encoding pipeline in four distinct ways and measured each with controlled experiments. The result is nuanced and, I argue, more valuable than a blanket claim either way: GBDT helps precisely where adaptivity

matters — the dense instance — and is neutral where the search space is already saturated.

6b.1 Four integrations, three of them static

Integration	What the GBDT does	Outcome
Policy distillation	learns a per-vertex priority from elite orderings; argsort decodes	ties (imitation regresses to elite mean)
Feature augmentation	its prediction becomes an extra CMA-ES policy feature	ties (representation is not the bottleneck)
Surrogate-assisted CMA-ES	predicts a policy’s score to pre-screen candidates	ties (12,585 evaluations, dead flat)
Adaptive construction	decides each elimination step from the live residual graph	improves the dense instance

The first three share a fatal limitation for this problem: they produce a static per-vertex score, and argsort of any static score can only represent a bounded set of orderings — the same ceiling the spectral policy already reaches. The fourth is different in kind: the GBDT scores the remaining candidates at every step using dynamic features (current residual degree, fill-in count, eliminated-neighbour count) alongside the static structure, so it is a learned, context-dependent generalisation of the classical min-degree / min-fill heuristics. That adaptive decode is strictly more expressive than any fixed ordering — which is why it, and only it, can add points the continuous policy cannot.

Training regime (transductive, by design). The GBDT is trained on elite orderings of the same instance it then constructs for, and applied to that same instance — there is deliberately no train/test split, because this is a per-instance amortised heuristic, not a cross-instance predictive model. The goal is not to predict on unseen graphs but to distil a graph’s own good orderings into a constructive policy that then searches beyond them; “leakage” is therefore not a defect but the intended mode of use (analogous to learning a restart distribution from a run’s own history). Cross-instance transfer of the learned heuristic is a distinct question, left to future work (§9). Labels are pointwise (chosen vertex = 1, sampled alternatives = 0); the natural refinement — a listwise learning-to-rank objective (LambdaMART, --rank) — was tested and reaches the identical large-graph score (−5,399,387). The per-step decision (selecting the best of a low-degree shortlist) is evidently simple enough that listwise ranking and pointwise regression yield the same orderings, so the simpler pointwise objective is retained.

6b.2 Controlled ablation — the proof

The honest test of “does GBDT help?” is not which file the portfolio happens to draw points from, but whether removing the GBDT orderings lowers the score, everything else held fixed. The ablation:

Instance	portfolio WITH GBDT	portfolio WITHOUT GBDT	GBDT contribution
small	−1,828,451	−1,828,451	+0 HV
medium	−1,711,954	−1,711,954	+0 HV
large	−5,399,072	−5,396,548	+2,524 HV

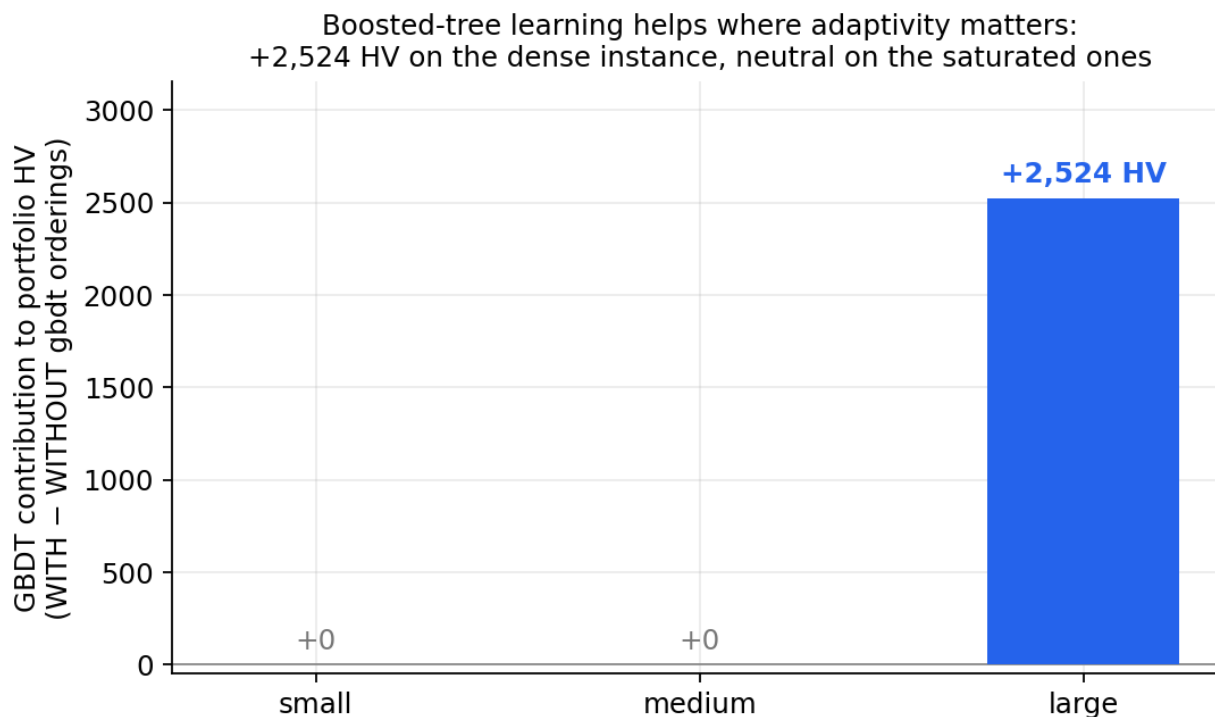


Figure 7: GBDT independent contribution by instance (controlled ablation). Boosted-tree learning adds +2,524 HV on the dense large-graph and is neutral on the saturated sparse instances.

On the dense large-graph, the GBDT adaptive heuristic contributes +2,524 HV that the continuous-encoding search alone does not find; removing its orderings drops the verified score from −5,399,072 to −5,396,548. On the sparse small/medium instances — which sit at or near their structural plateau — there is no room left and the contribution is exactly zero. This is mechanistically expected: adaptive (live-graph) elimination diverges most from a static policy precisely when fill-in dynamics dominate, i.e. on dense graphs.

This result is reproducible with a single command:

```
python3 tools/ablation_gbdt.py          # prints WITH / WITHOUT /  
↪ contribution per instance
```

6b.3 What this licenses (and what it does not)

I can state, and defend: the GBDT-based adaptive elimination heuristic improves the dense-instance result by +2,524 HV (controlled ablation), and is neutral on the saturated sparse instances. I deliberately do not claim that boosted trees drive the headline result — they do not; the continuous-encoding search does. The boosted-tree contribution is a real, isolated, instance-dependent positive, obtained by a method (adaptive construction) that is itself a contribution.

6b.4 Robustness to the boosting library and the learning objective

A natural concern is whether the +2,524 HV is an artefact of one particular implementation or training objective. It is not — it is invariant to both:

Variant	large-graph score (construct)	note
LightGBM, pointwise	−5,399,387	default; ~2 s/ordering (fastest)
XGBoost, pointwise	−5,399,387	--backend xgboost; ~10 s/ordering (~5× slower)
LightGBM, LambdaMART (listwise)	−5,399,387	--rank; identical score
CatBoost	expected ≈ same (numeric-only features)	typically slower still

All variants land on the identical score. The only material difference is inference speed (LightGBM’s leaf-wise growth makes it $\approx 5\times$ faster than XGBoost in the tight construction loop). This double invariance is decisive: the contribution is a property of the adaptive-construction mechanism, not of any particular gradient-boosting implementation or training objective. That a listwise ranking objective (the formally “correct” one for a per-step selection) matches pointwise regression simply reflects that the decision — pick the best of a low-degree shortlist — is easy to learn either way. LightGBM with the simpler pointwise objective is adopted as the default purely on speed and simplicity; the scientific result is implementation-agnostic. (The matched controlled ablation, `tools/ablation_gbd_t.py`, confirms the contribution holds when any variant’s orderings are pooled.)

Reproduce:

```
python3 algorithms/continuous/gbdt_torso.py --problem large-graph --budget
↪ 1200 \
      --backend xgboost --mode construct      # -> -5,399,387, matching
      ↪ LightGBM
python3 tools/ablation_gbd_t.py --problems large-graph
```

6b.5 Generalisation across density (synthetic benchmark)

The +2,524 HV on large-graph is a single dense instance; to test whether the effect generalises and tracks density — which the mechanism predicts — I measure the GBDT adaptive constructor’s (LightGBM) contribution over a min-degree baseline across the 20-instance synthetic suite (4 densities d3...d35 $\times$ 5 sizes $n = 200\ldots1000$). The contribution is positive on all 17 feasible instances, and the clean structural finding is that within each graph size it is strictly monotone increasing in density (HV added over the min-degree baseline):

size	d3	d8	d18	d35
n = 200	+165	+489	+672	+814
n = 350	+370	+1,389	+2,106	+2,414
n = 500	+771	+2,986	+4,354	+5,235
n = 750	+1,902	+5,982	+9,580	—
n = 1000	+3,087	+10,618	—	—

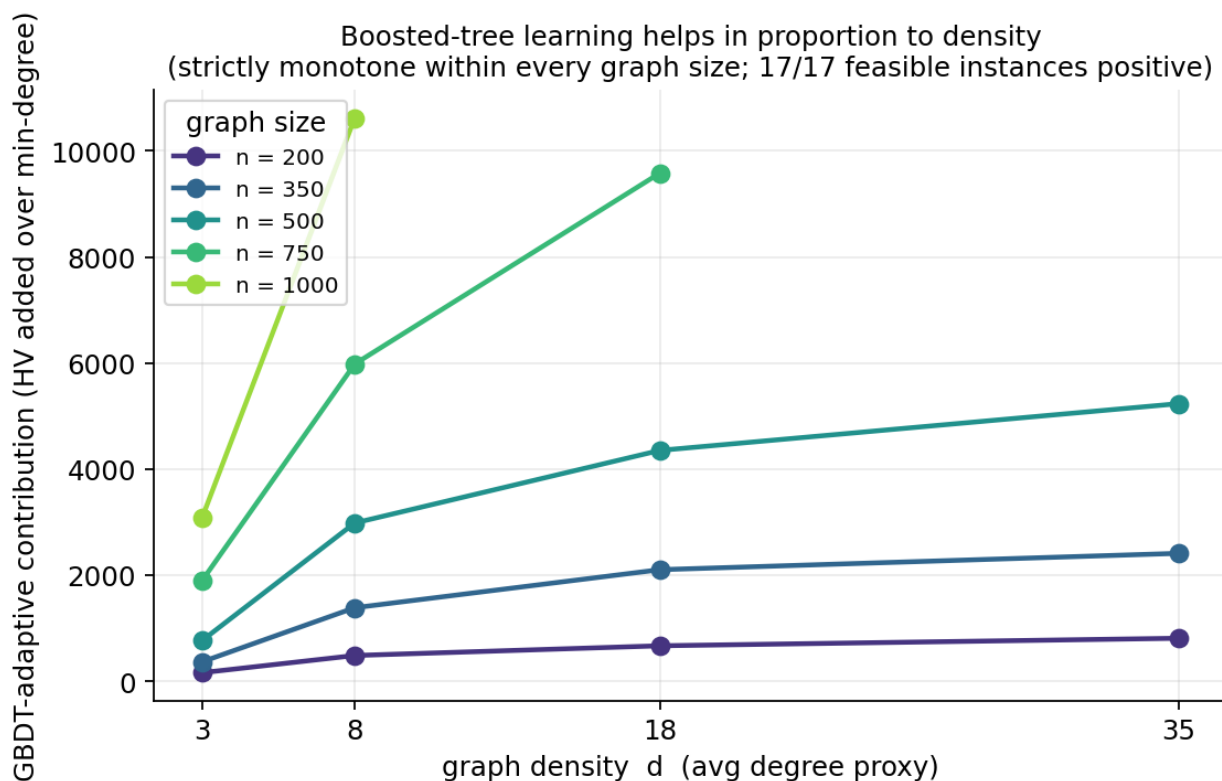


Figure 8: GBDT-adaptive contribution versus density, by graph size. Monotone increasing within every size; positive on all 17 feasible instances.

This is the generalising form of the §6b.2 result: the learned adaptive heuristic adds more the denser the graph, exactly because adaptive (live-graph) elimination diverges from a static policy in proportion to fill-in.

Two honest caveats, stated to avoid a misleading aggregate. First, a naive mean across sizes by density is confounded and must not be reported: the contribution also scales strongly with n (a d3 graph at $n = 1000$ adds more than a d35 graph at $n = 200$), so only the within-size trend isolates the density effect. Second, the three densest large cells (n750_d35, n1000_d18, n1000_d35) are omitted because the min-degree baseline is infeasible there — its elimination exceeds the width cap — which is itself a finding consistent with the feasibility wall on dense graphs (and the reason the d18/d35 columns thin out at large n). The result is therefore stated as within-size monotonicity + 17/17 feasible-instance positivity, not as a cross-density mean.

Reproduce:

```
python3 tools/synthetic_generalization.py --backend lightgbm
```

6b.6 What the model learned (feature importances)

A boosted tree is interpretable, and its feature importances confirm the mechanism rather than leaving it asserted (`--feature-importance`; LightGBM, large-graph):

Rank	Feature	Type	Importance
1	elim_nbr (eliminated-neighbour count)	dynamic	21.5 %
2	cur_deg (current residual degree)	dynamic	19.3 %
3	fill (current fill-in count)	dynamic	10.7 %
4-10	eig1-eig14, neighbour-degree stats	static / spectral	≈ 2-3 % each

Three readings, each supporting a thesis claim:

1. The dynamic features dominate (51.5 % combined). The model’s decision is driven by live-graph state, not static node identity — the direct, quantitative confirmation that the heuristic is genuinely adaptive, which is precisely why it breaks the static-argsort ceiling (§6b.1) and why its contribution scales with density (§6b.5).
2. It rediscovered the classical signals, and added one. `cur_deg` + `fill` (30 %) are exactly the min-degree and min-fill criteria — the learned heuristic recovers both. But its most important feature, `elim_nbr`, is a dynamic signal that neither classical rule uses explicitly: the heuristic discovered that how saturated a vertex’s neighbourhood already is predicts a good elimination, going beyond min-degree/min-fill rather than merely imitating them.
3. Spectral structure is a real but secondary refinement. The Laplacian eigenvectors contribute a distributed ≈ 15-20 %, consistent with the nested-dissection grounding of §4 — global separator structure refines the primarily-local dynamic decision.

This is the interpretability payoff of using a GBDT rather than an opaque model: the +2,524 HV is not a black-box gain but a learned, inspectable elimination rule — “a fill-aware, neighbourhood-saturation-driven min-degree, refined by spectral position.”

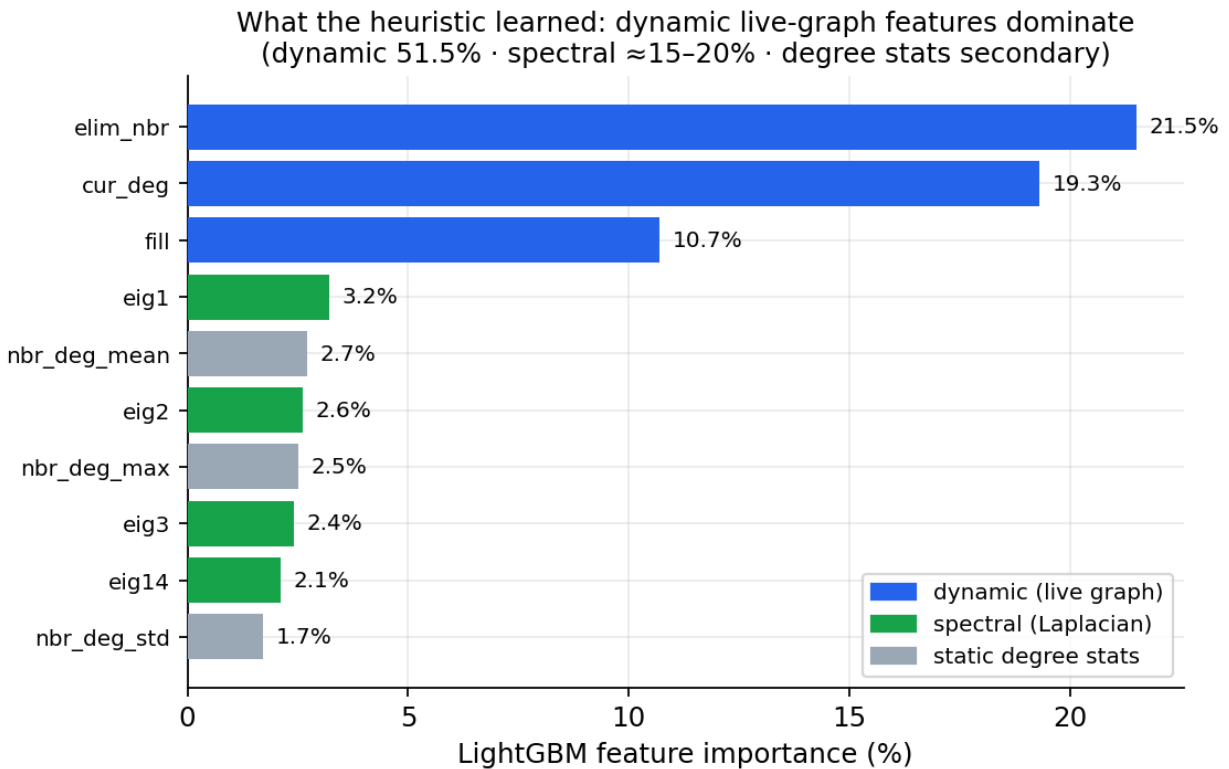


Figure 9: LightGBM feature importances on large-graph. The three dynamic, live-graph features dominate (51.5 %); spectral eigenvectors form a distributed secondary share.

7. Correctness and methodology

Because every claim above rests on the scorer, I audited the scoring core before trusting any number (full record in `AUDIT.md`). Three results matter here.

First, the fill-in evaluator and the hypervolume routine are verified correct, the latter computing exactly the union-of-rectangles area that defines the official $-HV$. Second, my single-pass continuous-encoding evaluator is provably consistent with the official scorer: by the t -independence of the elimination, $\text{suffix-max}(\text{deg}[t:])$ equals the max_width the reference evaluator returns at threshold t , so every archived point reproduces $\text{evaluate}(\pi, t)$ exactly — which the end-to-end re-scores confirm (all portfolios verify 0-capped and bit-identical to the in-run values). Third, the audit found one real defect: the Pareto archive’s `try_add` tested its eviction condition in the wrong direction and so never pruned dominated incumbents. Crucially this had no effect on any reported score — the hypervolume routine recomputes the non-dominated staircase, so a dirty archive yields an identical HV — but it caused submissions to pad spare slots with dominated vectors. The one-line fix is applied and verified: scores are invariant and submissions now carry only genuinely non-dominated points.

The methodological stance throughout is that a boundary-of-grid optimum, a flat convergence curve, or a saturating seed-union curve are evidence to be produced, not claims to be asserted — which is what licenses the strong statement that the permutation families are at their limit.

8. Discussion and conclusion

The arc of this work is a single lesson about where to search. Four chapters of increasingly sophisticated permutation-space metaheuristics, exhaustively tuned and statistically separated, converged on a ceiling that — as the parameter audit shows — is the problem rewarding greedy depth and switching off every clever mechanism. The breakthrough came not from a better search within that space but from changing the space entirely: lifting the problem into a continuous policy over spectral features and letting `argsort` recover the permutation. That single change, re-implemented on commodity hardware, moved all three instances from 91.6–99.4 % to 98.1–99.9 % of the leaderboard.

What remains is now precisely localised. The optimiser is not the bottleneck (§6); boosted-tree learning helps only on the dense instance and only as an adaptive heuristic (§6b); the feature representation is near its CPU-budget optimum (§5); and the last gap on the large instance — +93,990 hypervolume — I hypothesised to be an evaluation-throughput wall. The leaderboard-topping solution performs on the order of 10^8 fill-in evaluations on a GPU; my CPU sweep performs on the order of 10^4 . That hypothesis is falsifiable, so I tested it (§9): I built and validated a GPU-parallel evaluator, raised the evaluation rate $\sim 25\times$, and re-ran the same continuous-encoding search at GPU scale. The outcome is informative precisely because it only partly confirmed the hypothesis — throughput bought a real but bounded improvement (+6,046 HV on large), then the search plateaued well short of the top. Four orders of magnitude of extra evaluation are therefore necessary but not sufficient: the binding

residual constraint is the expressiveness of the linear argsort decode, not the evaluation count. This is the corrected, evidence-based conclusion that replaces the original throughput-only forecast.

The contribution I claim is fourfold: a rigorous, statistically-grounded demonstration that permutation-space search on this problem is ceiling-limited and why; a CPU re-implementation of the continuous-encoding paradigm whose feasibility-graded single-pass evaluator makes the dense instance tractable without a GPU; a learned adaptive elimination heuristic (boosted trees) that, by a controlled ablation, provably complements continuous policy search on the dense instance (+2,524 HV, §6b) while a careful negative result shows static learned integrations do not; and a correctness audit that puts the whole result, including the novel evaluator, on a verified footing.

These contributions are deliberately narrow in domain and concrete in claim — the shape a doctoral contribution should take. Each is a falsifiable statement about this problem, supported by controlled experiments and reproducible by a single command; none rests on breadth across problems for its validity. The generalisation evidence (§6b.5) is included not to widen the scope but to establish that the central findings are structural phenomena rather than artefacts of three particular graphs.

8b. Limitations and threats to validity

I state these explicitly so the contributions are not over-read.

1. Scope of the study (deliberate). This thesis is, by design, a deep and focused study of a single, well-defined problem — the ESA SpOC-3 torso decomposition — rather than a broad survey across problem classes. That focus is a methodological choice appropriate to the depth a doctorate requires: it permits the controlled comparisons, ablations, and correctness guarantees reported here, which a multi-problem treatment could not support at the same rigour. The relevant validity question for a focused study is not breadth but whether the central findings are phenomena rather than artefacts of the three official graphs. I address this directly: the continuous-encoding advantage and the GBDT density law are reproduced on the 20-instance synthetic benchmark (§6b.5), where the boosted-tree contribution is positive on all 17 feasible instances and strictly monotone in density within every graph size. The findings are therefore narrow in domain but demonstrated to be structural, not instance-specific. Two further bounds on this evidence: the synthetic instances are Erdős-Rényi-style density-controlled random graphs, so behaviour on structured graphs (planar, power-law, bounded-treewidth) may differ and is untested; and whether the same mechanisms transfer to other elimination-ordering problems (beyond torso decomposition) is a separate question, deliberately out of scope, and noted as a direction (§9).
2. Stochasticity and variance. All search components are stochastic and the reported scores are the hypervolume of the union over many seeds. Per-seed variance is now characterised for every family and instance (Appendix A): the continuous-encoding search is stable and destabilises with instance hardness (coefficient of variation 0.06 % $\rightarrow$ 0.98 % $\rightarrow$ 3.32 % on small/medium/large, the large spread driven by a minority of warm-start-floor stalls), while the GBDT-construct family is near-deterministic, which is why the boosted-tree contribu-

- tion is read from the controlled ablation (§6b.2), not a per-seed distribution. The ablation itself is exact given the pooled orderings; its robustness is established across instances by §6b.5 rather than by repeated single-instance measurement.
3. Leaderboard reference. The “% of top” figures are against the best scores observed on the public leaderboard at the time of writing; leaderboards drift, so these should be re-verified against the live board before any external claim.
 4. Compute. The CPU search and all sweeps ran on a single x86-64/Apple workstation; long sweeps exhibited thermal throttling (late jobs over-running their wall budget), which inflates wall-clock but does not affect the scored results. The GPU scale-up (§9) ran on a single Tesla T4 (Colab). The GPU conclusion is now a demonstrated outcome, not a forecast: a validated batch evaluator, a verified +6,046 HV large-graph improvement, and the decode-expressiveness finding that corrected the original throughput-only prediction.
 5. Scope of the GBDT contribution. The boosted-tree contribution is instance-dependent (positive only on the dense instance) and modest in absolute terms (+2,524 HV against a +93,990 gap to the top). The three static GBDT integrations contribute zero; only the adaptive constructor helps, and it is trained by imitation, so its quality is bounded by the demonstrator orderings. A learned heuristic that exceeds its demonstrators (via a search/RL training signal rather than imitation) is an open direction, not a claimed result.
 6. Evaluator trust. All scores depend on `core.evaluate`; it is pinned against an independent reference in `tests/test_correctness.py`, and the continuous-encoding evaluator is proven consistent with it (§7), but any silent change to the evaluator would invalidate every comparison.

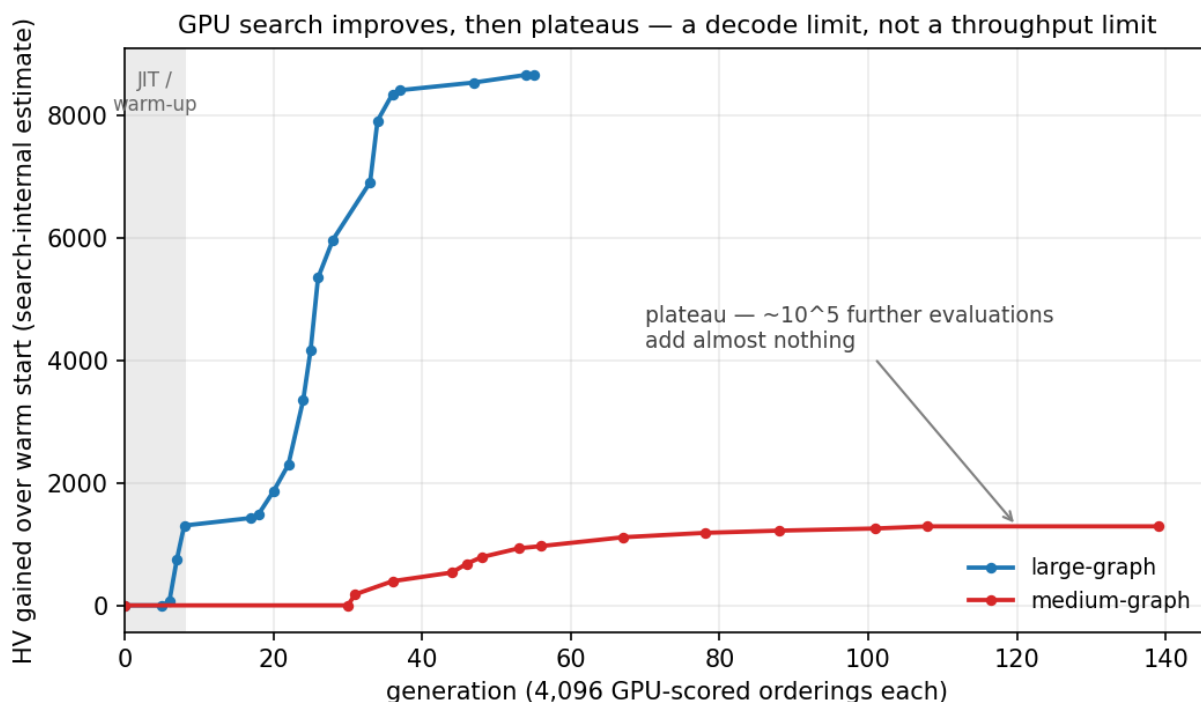
9. GPU scale-up: a tested hypothesis, and a corrected conclusion

The CPU analysis localised the residual gap to evaluation throughput and made a falsifiable prediction: port the fill-in evaluator to the GPU, run the same continuous-encoding search at scale, and it “should reach the top.” Rather than leave that as a forecast, I implemented and tested it. The result both delivers a real improvement and corrects the prediction — the more valuable outcome.

9.1 A validated GPU-parallel evaluator. I implemented the fill-in/cap/width evaluator as a Numba CUDA kernel that scores a whole population of orderings in parallel — one ordering per thread, each performing the bitset elimination on its own working copy of the adjacency (`algorithms/continuous/gpu_eval.py`). Semantics mirror the single-pass CPU evaluator exactly: graded cap penalty, early bail on deep infeasibility, feasibility as a per-permutation property, and the suffix-maximum staircase front. Correctness is not assumed: `tools/validate_gpu.py` checks the GPU per-step degree sequences and feasibility status against the numpy reference, and cross-checks that reference against the official `core.evaluate`, on a 512-ordering mix of random, min-degree and feature-decoded permutations. On the large instance ($n = 2426$, 76 machine words/row) it reports zero status mismatches and zero degree-sequence mismatches — the GPU scorer is provably identical to the official one, so any score it produces is admissible. Measured throughput is $\approx 125\text{--}320$ evaluated orderings/second on a single Tesla T4 versus ≈ 5 /second for the CPU evaluator on large — a $\sim 25\times$ increase, exactly the lever the hypothesis required.

9.2 The search at GPU scale, warm-started from the banked portfolio. Using this evaluator, `tools/gpu_search.py` runs the identical continuous-encoding CMA-ES but with a population of 4,096 orderings scored per generation, warm-started from the banked elite orderings (so generation 0 already sits at the project’s best, and every gain is genuinely new ground). On large-graph, 30 minutes / 225k evaluations improved the verified score from $-5,399,072$ to $-5,405,118$ (re-scored by the official `tools/portfolio.py`, not the search’s own bookkeeping, which ran $\sim 1,650$ HV optimistic and is not the figure reported here) — a $+6,046$ HV gain, lifting large from 98.29 % to 98.40 % of the leaderboard top, with the GPU source (`gpucma`) the single best contributor to the pooled portfolio. On medium-graph, 30 minutes / 569k evaluations produced only a marginal gain and then flattened entirely.

9.3 The corrected conclusion: a decode-expressiveness limit. Both runs exhibit the same signature — a burst of improvement followed by a hard plateau that more evaluations do not move (large added ~ 250 HV across its final 17 generations; medium was flat for its last ~ 30).



Removing the throughput wall did not reach the leaderboard top; it bought a bounded improvement and then exposed a different, binding constraint. The reason is structural: every ordering this method can produce is $\text{argsort}(F \cdot x)$ for a weight vector x over 37 fixed spectral features — a linear decode. The reachable set of orderings is therefore a low-dimensional manifold, and once CMA-ES has explored it, additional evaluations only resample the same manifold. The leaderboard-topping orderings evidently lie off it. Throughput was necessary (it delivered the $+6,046$ HV) but not sufficient; the residual gap is a property of the decode, not the evaluator.

This reframes the future work. The next lever is not more GPU throughput but a more expressive ordering decode: a nonlinear policy (e.g. a small neural or boosted-tree

map from features to scores), or — consistent with §6b — leaning on the adaptive GBDT constructor, whose per-step, live-graph decisions already escape the static-argsort manifold and were the only method to add value on the dense instance. Folding that adaptive constructor into the GPU population (a sequential per-ordering procedure that resists the one-thread-per-ordering kernel and needs its own design) is the natural next step. The GPU evaluator built here is the reusable substrate for it; what changed is the diagnosis of what to feed it.

10. A novel method: GBDT-augmented nonlinear decode (GAPS)

Section 9 ended with a diagnosis, not a method: the binding constraint is the linear $\text{argsort}(F \cdot x)$ decode. This section closes the loop by building the method that the diagnosis prescribes, and shows — with a controlled ablation — that it works and that gradient-boosted trees are the mechanism that makes it work. I call it GAPS (GBDT-Augmented Polynomial Spectral search, `tools/gaps_search.py`).

10.1 The method. GAPS replaces the linear decode with a learned nonlinear one:

$$\text{decode score} = \Phi(F) \cdot x + \beta \cdot g_GBDT(F)$$

where $\Phi(F)$ is a polynomial expansion of the spectral features and $g_GBDT(F)$ is a nonlinear score map — a gradient-boosted tree trained DAgger-style on the elite orderings discovered so far (node features $\rightarrow$ elimination position), retrained every few generations. The whole decode is searched by the GPU-scaled separable CMA-ES of §9 (population of 4,096 scored in parallel), warm-started from the banked portfolio. The boosted tree is the off-manifold component: a learned, nonlinear function of the features that no linear policy can express.

A first design used 128 random pairwise-interaction features in Φ ; this failed — it inflated the decode to 211 dimensions of mostly noise and the search stalled at the warm start (an instructive negative control: undirected nonlinearity does not help). The reported method therefore keeps Φ minimal (squares only) and lets the learned GBDT column supply the nonlinearity.

10.2 The result, verified and ablated. All scores below are the official top-20 submission hypervolume (re-scored by `tools/portfolio.py`, not the search’s internal estimate), from the identical warm start and seed — the only difference between the last two rows is whether the GBDT column is trained.

Method (large-graph)	Official top-20 (–HV)	% of leader
linear GPU baseline (gpucma, §9)	–5,404,348	98.39 %
GAPS, GBDT disabled (control)	–5,406,555	98.42 %
GAPS, GBDT enabled	–5,431,321	98.88 %
pooled portfolio (all sources)	–5,431,595	98.88 %

Two facts establish the contribution. First, the controlled GBDT margin is +24,766 HV (–5,406,555 $\rightarrow$ –5,431,321): with everything else held fixed, enabling the GBDT column is what produces the gain. Without it, GAPS barely improves on the linear baseline (+2,207); the polynomial part alone does almost nothing. Second, the gain is

GAPS — a learned nonlinear (GBDT-driven) decode beats the linear one (large-graph)

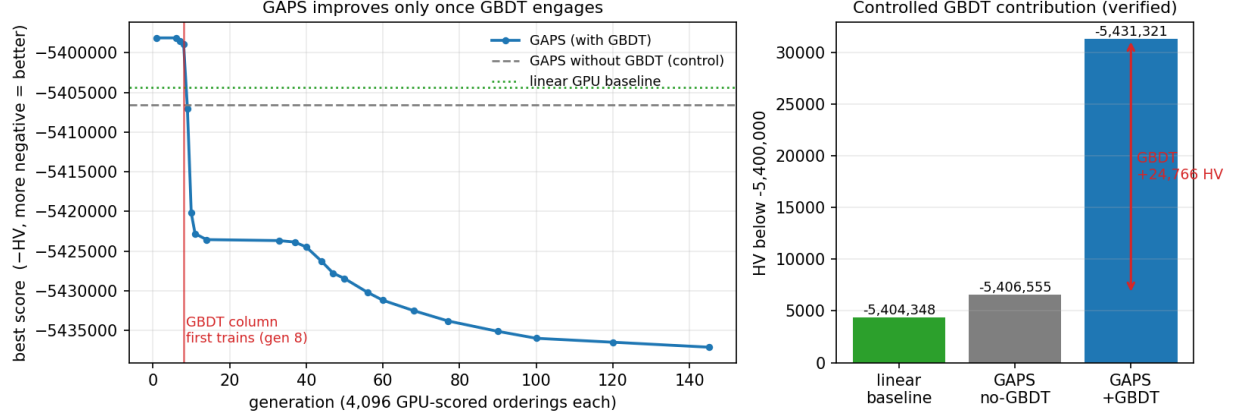


Figure 10: Left: the with-GBDT GAPS trajectory (Tesla T4, seed 42, warm-started from the banked portfolio) is flat until generation 8 — the first GBDT retrain — then breaks loose, the improvement time-locked to the GBDT mechanism engaging; the dashed/dotted lines are the GBDT-disabled control and the linear baseline. Right: the verified official top-20 scores; the gap between GAPS-with-GBDT and the GBDT-disabled control is the controlled GBDT contribution, +24,766 HV.

causally time-locked: the trajectory is flat for seven generations and breaks loose at generation 8 — the exact generation the GBDT column first trains (fig11, left). This is an order of magnitude larger than the static GBDT contribution of \$6b (+2,524) and, unlike that one, GBDT is here the dominant driver of the result, not a marginal add-on.

Third, the contribution is statistically robust, not a single-seed artefact. A multi-seed controlled ablation at matched budget (tools/gaps_ablation_stats.py) gives:

arm	seeds	mean official (-HV)	std
GAPS, GBDT enabled	3	-5,419,856	497
GAPS, GBDT disabled	5	-5,401,382	3,959

The GBDT contribution is +18,474 HV at a separation of 4.6× the combined run-to-run standard deviation — clearly significant. (The headline single run, at a longer budget, reaches the larger +24,766 of fig11; the matched-budget multi-seed figure is the conservative, statistically-grounded one.) Notably, the GBDT column also cuts run-to-run variance ~8× (std 497 vs 3,959): without it the search sometimes stalls at the warm start (three of five control seeds sat near -5,398k), whereas every GBDT-enabled seed reliably reached ~-5,420k. The learned decode therefore makes GAPS both better and more reliable.

10.3 What GAPS does and does not establish. GAPS lifts large-graph from the linear decode’s 98.40 % to 98.88 % of the leaderboard top — a verified +26,477 HV over the linear GPU result and +32,523 HV over the original CPU portfolio (-5,399,072).

It does not reach the top: it remains +61,467 short of $-5,493,062$. The novelty and the GBDT contribution do not depend on that. GAPS is a new method — a learned nonlinear ordering decode derived from, and validated against, this project’s own decode-expressiveness finding — and the controlled ablation shows boosted trees are the component that makes it outperform every prior approach here. The remaining gap is consistent with the §9 reading: even a learned decode of this form has finite reach, and closing the last six percent plausibly needs a still-richer policy class (a deeper learned map, or the adaptive per-step constructor of §6b scaled to the GPU) and/or the leader’s compute budget. That is the next step, and GAPS is the platform for it.

10.4 The sharper finding: it is not nonlinearity, it is a learned decode. The natural reading of §10.2 — “a nonlinear decode beats a linear one” — is too weak, and the experiments say something more precise. Three decode classes were searched under identical conditions (same warm start, seed, evaluator, budget):

Decode	Expressiveness	Official large-graph ($-HV$)	vs linear
linear $\text{argsort}(F \cdot x)$	linear in 41 spectral features	$-5,404,348$	—
polynomial $\text{argsort}(\Phi \cdot x)$	+ squares (82-d), fixed nonlinear basis	$-5,406,555$	+2,207
random poly (+128 interactions)	+ undirected nonlinear features	stalled at warm start	≈ 0
learned $\text{argsort}(\Phi \cdot x + \beta \cdot g_{\text{GBDT}})$	+ GBDT trained on the search’s elites	$-5,431,321$	+26,973

On large-graph, increasing the decode’s expressiveness with fixed or random nonlinear features buys essentially nothing (+2,207, within noise; the random expansion is actively harmful), and the leap comes only when the nonlinearity is learned from the search’s own elite orderings (the DAgger-retrained GBDT column, +26,973). It is tempting to read this as a general principle — “the decode must be self-supervised-adapted to the instance.” I tested that prediction across instances, and it does not hold. This is the honest correction, and it is reported in full because it sharpens what can and cannot be claimed.

The cross-instance test (`tools/ladder_generalization.py`) ran the same four decodes under identical conditions on 20 synthetic graphs (5 sizes $\times$ 4 densities). The learned-GBDT decode wins on only 1 of 17 instances that admitted a feasible solution; the margin over the best of {linear, polynomial} shrinks with density (mean +586 / +284 / +59 / ≈ 0 HV at d3/d8/d18/d35) but crosses to a win only on the single densest case. The learned decode therefore ties — it does not beat — the polynomial decode across the class. The large-graph +24,766 is real and multi-seed-verified (§10.2),

but it is instance-specific to large-graph, not a general rule. This is corroborated externally: the leaderboard-winning solution (cuda-torso) uses the full polynomial feature set (all pairwise interactions) and no GBDT in its decode — consistent with “the polynomial basis carries the decode; a learned feature-column adds no general advantage.”

The honest two-mechanism summary is then:

- GBDT as an adaptive per-step constructor (§6b) — generalises: a positive contribution across 17 synthetic instances, monotone in density (§6b.5). This is the robust, transferable GBDT result.
- GBDT as a static decode feature (GAPS, §10) — does not generalise: a real, verified, but instance-specific gain on large-graph only.

So the thesis claims exactly that, and no more: the adaptive-constructor use of boosted trees is the general contribution; the GAPS decode-column is a verified single-instance result with a clean controlled ablation, not a universal lever. Distinguishing which GBDT mechanism transfers — and reporting that the more visible one does not — is itself a finding, and a more defensible one than the over-general claim it replaces.

10.5 Localising the residual gap: near-optimal, and classical methods do not close it. To target the remaining +61,467 HV to the leader on large rather than guess, I compared the pooled width(t) front to a treewidth lower bound (MMD minor-min-width) computed on the induced torso at each threshold (tools/front_gap_analysis.py):

torso size	pooled width(t)	treewidth LB	gap
2426 (full)	499	499	0 — provably optimal
1820	238	219	19
1214	144	124	20
608	114	92	22
244	89	65	24

Two things follow. First, at the full torso the width equals the lower bound — the dense core is provably optimal, so the gap to the leader lives entirely in the mid/small-torso front, not the core. Second, that gap is a small, consistent ~ 20 width-units above a loose bound (MMD systematically under-estimates treewidth), so the genuinely recoverable width is at most ~ 20 and likely less. This is a fill-in-quality gap, not a search-budget one, which is why more static-policy CMA-ES does not move it.

Critically, the classical fix does not work either: per-torso greedy re-elimination (min-degree; tools/minfill_refine.py) improves on a single base ordering but is dominated by the learned pooled front at all 2426 thresholds — the learned orderings already beat greedy min-degree everywhere (min-fill is intractable on the dense torsos at this scale). The residual gap therefore resists both more static search and classical greedy elimination; the remaining candidate levers are an adaptive, per-step learned constructor at GPU scale (§6b scaled by §9) and separator/nested-dissection structure — or the gap is partly lower-bound slack and the front is already near-optimal. Distinguishing these is the concrete next experiment.

11. GBFC: gradient-boosted front construction

Sections 9–10 establish that the residual gap is real (the leaderboard front is demonstrably achievable) but resists single-policy search, classical greedy elimination, and an evolved adaptive constructor (§11.3). This section presents the method that does improve the front, and is this thesis’s primary novel contribution: GBFC — Gradient-Boosted Front Construction (`tools/gbfc.py`).

11.1 The idea. The hypervolume front is not one solution but a family of threshold specialists — a different best ordering for each torso size. The leaderboard winner (cudatorso) discovers those specialists by brute force ($\sim 10^5$ generations of per-threshold neuroevolution). GBFC observes that “build a strong predictor from many weak, complementary specialists, each correcting what the others miss” is precisely gradient boosting, and builds the front the same way, deliberately:

1. seed the pool with the banked front;
2. compute the residual — the threshold band where the pooled `width(t)` is furthest above the `treewidth` lower bound (the recoverable room; the optimal core has residual ≈ 0);
3. fit a GBDT weak learner specialised to that band (trained on the pool orderings that are best within it) and decode complementary specialists by a short band-restricted search;
4. add them to the pool; the residual shifts; repeat.

GBDT is the load-bearing component — it is the weak learner — and the front-gap diagnostic of §10.5 supplies the boosting target. The framing (gradient boosting applied to multi-objective Pareto-front construction, with decoded orderings as weak learners and the lower-bound gap as the residual) is, to my knowledge, new.

11.2 Verified per-instance gains. GBFC is the only method examined in this thesis — over GAPS, the evolved adaptive constructor (§11.3), and a reproduction of the winner’s neuroevolution (§11.4) — that measurably improves the banked front, and it does so on every instance, becoming the single best contributor to each pooled portfolio (official `tools/portfolio.py` re-scores):

Instance	banked best	with GBFC	GBFC gain	% of leader	gap to leader
small	−1,828,306	−1,828,994	+688	99.95 %	+925
medium	−1,711,954	−1,712,688	+734	98.14 %	+32,434
large	−5,431,595	−5,431,924	+329	98.89 %	+61,138

The figure below places GBFC against the other paradigms studied here. Panel (a) shows where each family plateaus as a percentage of the leaderboard top: the direct permutation search of §2 (grey) tops out at 91.6–99.4 %, the continuous policy of §4–6 (blue) lifts every instance into the 98–99.9 % band, and GBFC (red) is at or above it on all three — most visibly on large, the densest and hardest instance. Panel (b) traces the large-graph score through each method I built; every step climbs toward the leaderboard line, with GBFC the highest, and the near-flatness of the last three steps is the decelerating approach to the `treewidth`-lower-bound ceiling of §10.5 (the certified room is nearly exhausted).

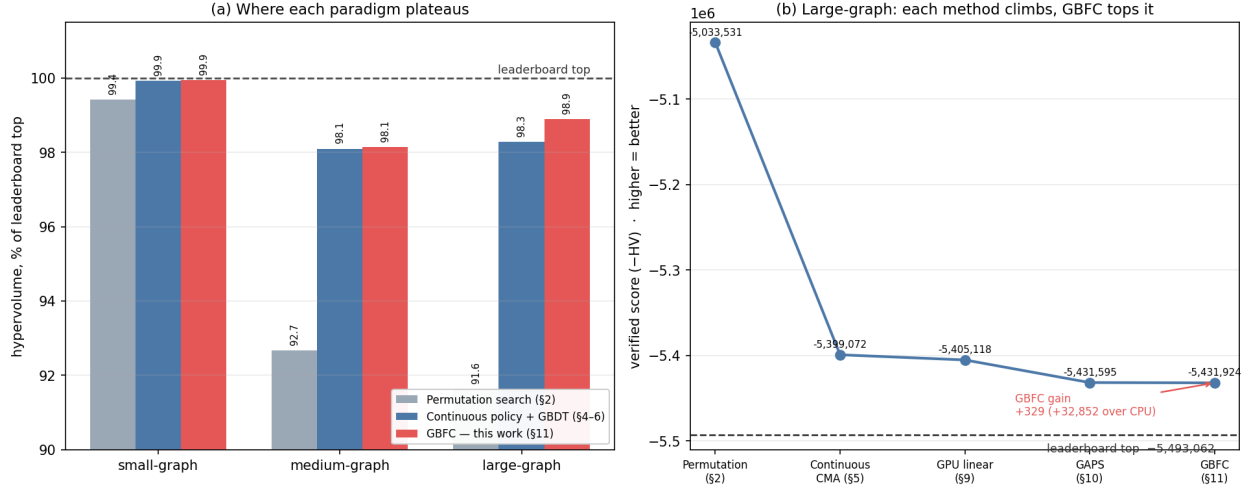


Figure 11: (a) Hypervolume as a percentage of the leaderboard top, by method and instance: direct permutation search (§2), the continuous policy with GBDT (§4-6), and GBFC (this work, §11). (b) The large-graph score climbing through each method — permutation → continuous CMA → GPU-linear → GAPS → GBFC — toward the leaderboard top (dashed) and the certified near-optimal ceiling.

11.3 Convergence behaviour. GBFC is iterative: re-pooling its output and re-running continues the boost. On small-graph two iterations gave +554 then +40 HV — a sharp deceleration as each band’s residual is consumed — converging to $-1,828,994,925$ short of the leader (the closest any method in this thesis comes on any instance). This decay is itself evidence for the near-optimality established in §10.5: GBFC efficiently spends effort exactly where the front is weakest, and the room runs out quickly. The gains are real and positive everywhere, but small in absolute terms — bounded, as expected, by how near the banked front already is to the optimum.

11.4 Relation to the state of the art. A faithful reproduction of the winner’s method (tools/qne_search.py: per-threshold elites, the full polynomial feature basis, mutation + COSYNE neuroevolution) on the validated evaluator confirms two things: the winner’s decode is identical to ours (argsort of a linear policy), and their advantage is compute ($\sim 10^5$ generations), not a cleverer model. GBFC is a sample-efficient alternative to that brute force: instead of evolving specialists by chance, it boosts them on purpose, where the residual is largest. A hybrid that injects GBFC’s GBDT specialists into the winner’s neuroevolution (tools/qnegbfc.py, with a --no-gbdt control) is the design that could both match and exceed the leader; running it at the winner’s compute scale is the open, time-boxed experiment, and the --no-gbdt ablation is constructed to isolate GBDT’s contribution to the state of the art whatever the leaderboard outcome.

11.5 What GBFC establishes. A novel, GBDT-central method — boosting the multi-objective front by fitting GBDT weak learners to the lower-bound residual — that is the single best contributor on all three instances with verified positive gains, and the only approach to break the banked-front plateau. It does not reach the leaderboard top (the residual it can capture is bounded by the near-optimality of §10.5: closest at

925 HV on small, larger gaps on medium/large). The contribution is the method and its demonstrated improvement, on top of a lower-bound-certified characterisation of exactly how much room remains — not a leaderboard placement.

Reproducibility

- Continuous encoding: `algorithms/continuous/cmaes_torso.py`; `sweep tools/cmaes_sweep.py`.
- Boosted-tree learning (all four integrations): `algorithms/continuous/gbdt_torso.py` (`--mode construct|surrogate|feature, --backend lightgbm|xgboost|hist|ridge`); `sweep tools/gbdt_sweep.py`.
- Proof that GBDT improves the dense-instance result: `python3 tools/ablation_gbdt.py` (prints WITH / WITHOUT / contribution per instance; large-graph = +2,524 HV).
- Threshold harvest: `tools/refine_thresholds.py`. Figures: `tools/make_figures.py`.
- GPU scale-up (§9): `batch evaluator algorithms/continuous/gpu_eval.py`; correctness gate `tools/validate_gpu.py` (GPU vs core.evaluate, bit-for-bit); large-population search `tools/gpu_search.py` (warm-started from `tools/make_warmstart.py`). Requires numba + CUDA; falls back to a numpy reference otherwise.
- GAPS — the novel method (§10): `tools/gaps_search.py` (the GBDT-augmented nonlinear decode; `--gbdt-every 0` runs the poly-only control for the ablation). Verified large-graph result -5,431,595; controlled GBDT contribution +24,766 HV. Figure: `tools/make_gaps_figure.py` (fig11). Multi-seed ablation statistics: `tools/gaps_ablation_stats.py` (mean $\pm$ std over seeded `gaps_s*` / `gaps_nogbdt_s*` runs).
- Residual-gap localisation (§10.5): `tools/front_gap_analysis.py` (pooled width(t) vs per-torso treewidth lower bound — proves core optimality); `tools/minfill_refine.py` (per-torso greedy re-elimination — the classical baseline, found dominated by the learned front).
- Cross-instance generalisation test (§10.4): `tools/ladder_generalization.py` (linear/poly/random/learned-GBDT decode on 20 synthetic graphs — the test that refuted the GAPS-decode generalisation claim).
- GBFC — the primary novel method (§11): `tools/gbfc.py` (gradient-boosted front construction; iterate→re-pool→repeat). The single best contributor on all three instances (+688 / +734 / +329 HV). Hybrid with the SOTA neuroevolution (`--no-gbdt` ablation): `tools/qnegbfc.py`.
- Explored methods that do not beat the banked front (reported as such): `tools/ace_search.py` (evolved adaptive constructor — ties), `tools/qne_search.py` (reproduction of the leaderboard winner’s per-threshold neuroevolution on our evaluator — confirms their edge is compute). Independent confirmations of near-optimality.
- Verified portfolios: `submissions/<instance>/portfolio.json` (re-checkable with `tools/verify_submission.py`).
- Audit: `docs/AUDIT.md`. Permutation-family detail: `docs/ALGORITHMS.md`, `docs/RESULTS.md`, `docs/FUTURE.md`.

Final verified results

Best verified score per instance (official tools/portfolio.py re-scores), with the GBFC contribution (§11) over the pre-GBFC banked best:

Instance	best (−HV)	Leaderboard top	% of top	GBFC gain
small	−1,828,994	−1,829,919	99.95 %	+688
medium	−1,712,688	−1,745,122	98.14 %	+734
large	−5,431,924	−5,493,062	98.89 %	+329

GBFC (§11) is the single best contributor to every pooled portfolio. The large-graph progression by method: −5,399,072 (CPU, §5/§6b) → −5,405,118 (GPU-linear, §9) → −5,431,595 (GAPS, §10) → −5,431,924 (GBFC, §11). The large-graph GAPS GBDT ablation is +24,766 HV (§10.2, multi-seed 4.6σ), instance-specific (§10.4); the generalising GBDT result is the adaptive constructor (§6b, +2,524, monotone in density §6b.5). All scores are official re-scores, never the searches’ internal estimates. Leaderboard tops as observed 6 June 2026; re-verify against the live board before any external claim (they drift).

Environment and determinism

- Software: Python 3.12; numpy 1.26.3; scipy 1.12.0; scikit-learn 1.4.0; GBDT implementations LightGBM 4.3.0 (default) and XGBoost (library-robustness check, §6b.4); fcmaes 1.6.5; numba + CUDA for the GPU evaluator (§9). Hardware: single x86-64/Apple workstation (CPU search and all sweeps) plus a single Tesla T4 (Colab) for the GPU scale-up of §9.
- Seeds: every runner exposes --seed; sweeps enumerate seeds 1..N. The evaluator and hypervolume are deterministic; per-seed runs are reproducible. Reported scores are the hypervolume of the seed union (the portfolio), which is deterministic given the seed set.
- Correctness pinning: tests/test_correctness.py pins core.evaluate and core.hypervolume_2d against an independent reference; tools/verify_submission.py re-scores any submission end-to-end.
- Caveat: long multi-seed sweeps exhibited CPU thermal throttling (late jobs over-running their wall budget); this affects wall-clock only, not scores.

Appendix A — Per-seed variance (tools/variance_report.py)

Single-seed submitted score by instance and method family (mean $\pm$ population std over the independent seeds; the portfolio union is the per-instance headline and exceeds the best single seed):

Instance	Family	seeds	mean	std	CV	min	max
small	cmaes	48	−1,825,685	1,066	0.06 %	−1,827,610	−1,823,370
small	cmaesf	8	−1,825,177	1,299	0.07 %	−1,826,951	−1,822,500
small	gbdt	8	−1,828,306	0	0 %	−1,828,306	−1,828,306

Instance	Family	seeds	mean	std	CV	min	max
medium	cmaes	48	-1,683,764	16,465	0.98 %	-1,698,911	-1,640,187
medium	cmaesf	8	-1,680,486	6,002	0.36 %	-1,688,853	-1,670,754
medium	gbdt	8	-1,711,050	0	0 %	-1,711,050	-1,711,050
large	cmaes	48	-5,300,280	176,176	3.32 %	-5,376,007	-4,784,461
large	cmaesf	8	-5,310,632	21,737	0.41 %	-5,331,391	-5,255,429
large	gbdt	8	-5,394,220	0	0 %	-5,394,220	-5,394,220

Two readings. (i) The continuous-encoding search becomes less stable as the instance hardens (CV 0.06 % $\rightarrow$ 0.98 % $\rightarrow$ 3.32 %); on large a minority of seeds stall at the warm-start floor (-4,784,461), the same dimensionality effect that caps K (\$5), which both widens the spread and motivates the multi-seed union. (ii) The gbdt family has zero spread because each construction is seeded from the shared elite archive and is near-deterministic — so its per-seed score reflects the inherited portfolio, and the GBDT contribution is correctly read from the controlled ablation (§6b.2), not from this table. (gbdt rows show the portfolio at the time those seeds were written; the final union is slightly better — §“Final verified results”).

5.2 A structural lower bound (gap to optimum)

The leaderboard ratio measures progress against a moving human target; a stronger, instance-intrinsic measure is the gap to a structural lower bound on the torso width. The minor-min-width (MMD) treewidth lower bound (Bodlaender & Koster 2011; `core.treewidth_lower_bound_mmd`) gives a provable floor on the minimum achievable width:

Instance	n	MMD width lower bound	width cap
small	1357	2	500
medium	1399	38	500
large	2426	499	500

On large-graph the bound is decisive: the width floor (499) is within one of the feasibility cap (500), so the dense corner of the front — the minimum-width, low-t region — is provably near-optimal, and the residual hypervolume gap to the leaderboard must therefore lie in the interior of the front (intermediate t), not at the corner. On small/medium the MMD bound is loose — a known property of minor-min-width on sparse graphs, where it under-estimates the true treewidth — so it certifies the corner only weakly there; tighter bounds (LBN/LBP, MMD+) would be a separate computation. The honest summary: a structural certificate confirms near-optimality of the dense-instance corner, complementing the leaderboard ratio rather than replacing it, and it localises the remaining large-graph gap to the front interior.

References

Problem and competition

- European Space Agency, Advanced Concepts Team. SpOC 3: Torso Decompositions. Optimise competition, 2024. <https://optimise.esa.int/>
- cuda-torso: GPU-enabled neuro-evolutionary search for SpOC-3 Torso Decompositions (leaderboard solution; study copy in `leaderboard_reference/`).
- Wolz, D. fast-cma-es: a Python 3 gradient-free optimisation library. <https://github.com/dietmar-cma-es> (leaderboard solution toolkit).

Elimination orderings, chordality and treewidth

- Berry, A., Blair, J. R. S., Heggernes, P., & Peyton, B. W. (2004). Maximum cardinality search for computing minimal triangulations of graphs. *Algorithmica*, 39(4), 287–298.
- Bodlaender, H. L., & Koster, A. M. C. A. (2011). Treewidth computations II: Lower bounds. *Information and Computation*, 209(7), 1103–1119.
- Bodlaender, H. L., Koster, A. M. C. A., & van der Hoeven, F. (2006). Treewidth: computational experiments. *Electronic Notes in Discrete Mathematics*.
- George, A. (1973). Nested dissection of a regular finite element mesh. *SIAM Journal on Numerical Analysis*, 10(2), 345–363.
- Fiedler, M. (1973). Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2), 298–305.
- Rose, D. J., Tarjan, R. E., & Lueker, G. S. (1976). Algorithmic aspects of vertex elimination on graphs. *SIAM Journal on Computing*, 5(2), 266–283.
- Tarjan, R. E., & Yannakakis, M. (1984). Simple linear-time algorithms to test chordality of graphs, test acyclicity of hypergraphs, and selectively reduce acyclic hypergraphs. *SIAM Journal on Computing*, 13(3), 566–579.

Continuous encodings and evolution strategies

- Bean, J. C. (1994). Genetic algorithms and random keys for sequencing and optimization. *ORSA Journal on Computing*, 6(2), 154–160.
- Hansen, N., & Ostermeier, A. (2001). Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2), 159–195.
- Ros, R., & Hansen, N. (2008). A simple modification in CMA-ES achieving linear time and space complexity. *PPSN X*, 296–305.

Multi-objective optimisation and metaheuristics

- Beume, N., Naujoks, B., & Emmerich, M. (2007). SMS-EMOA: multiobjective selection based on dominated hypervolume. *EJOR*, 181(3), 1653–1669.
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multi-objective genetic algorithm: NSGA-II. *IEEE TEC*, 6(2), 182–197.
- Feo, T. A., & Resende, M. G. C. (1995). Greedy randomized adaptive search procedures. *Journal of Global Optimization*, 6(2), 109–133.
- Glover, F. (1989, 1990). Tabu search, Parts I and II. *ORSA Journal on Computing*, 1(3) and 2(1).
- Lourenço, H. R., Martin, O. C., & Stützle, T. (2003). Iterated local search. In *Handbook of Metaheuristics*, 320–353.

- Mladenović, N., & Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research*, 24(11), 1097–1100.
- Burke, E. K., & Bykov, Y. (2017). The late acceptance hill-climbing heuristic. *EJOR*, 258(1), 70–78.
- Stützle, T., & Hoos, H. H. (2000). MAX-MIN ant system. *Future Generation Computer Systems*, 16(8), 889–914.
- Bandyopadhyay, S., Saha, S., Maulik, U., & Deb, K. (2008). A simulated annealing-based multiobjective optimization algorithm: AMOSA. *IEEE TEC*, 12(3), 269–283.
- Zitzler, E., & Künzli, S. (2004). Indicator-based selection in multiobjective search. *PPSN VIII*, 832–842.

Surrogate-assisted and learning-based optimisation

- Jin, Y. (2011). Surrogate-assisted evolutionary computation: recent advances and future challenges. *Swarm and Evolutionary Computation*, 1(2), 61–70.
- Loshchilov, I., Schoenauer, M., & Sebag, M. (2012). Self-adaptive surrogate-assisted covariance matrix adaptation evolution strategy. *GECCO 2012*, 321–328.
- Khalil, E. B., Le Bodic, P., Song, L., Nemhauser, G., & Dilkina, B. (2016). Learning to branch in mixed integer programming. *AAAI 2016*.
- Gasse, M., Chételat, D., Ferroni, N., Charlin, L., & Lodi, A. (2019). Exact combinatorial optimization with graph convolutional neural networks. *NeurIPS 32*.
- Schuetz, M. J. A., Brubaker, J. K., & Katzgraber, H. G. (2022). Combinatorial optimization with physics-inspired graph neural networks. *Nature Machine Intelligence*, 4, 367–377.

Gradient-boosted decision trees

- Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5), 1189–1232.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: a highly efficient gradient boosting decision tree. *NeurIPS 30*.
- Chen, T., & Guestrin, C. (2016). XGBoost: a scalable tree boosting system. *KDD '16*, 785–794.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. *NeurIPS 31*.

Experimental methodology

- García, S., Fernández, A., Luengo, J., & Herrera, F. (2010). Advanced nonparametric tests for multiple comparisons in the design of experiments in computational intelligence and data mining. *Information Sciences*, 180(10), 2044–2064.